Cloud Coachers

DEV450 - Salesforce.com Platform Developer I

salesforce

2022

Module 5:
DATA MANIPULATION LANGUAGE

Module 5: Data Manipulation Language

- **Insert & Update**
- How DML Works
  - Database Class
  - DML Handling
- Related Records
- Others DMLs
  - Upsert
  - Delete & Undelete
-



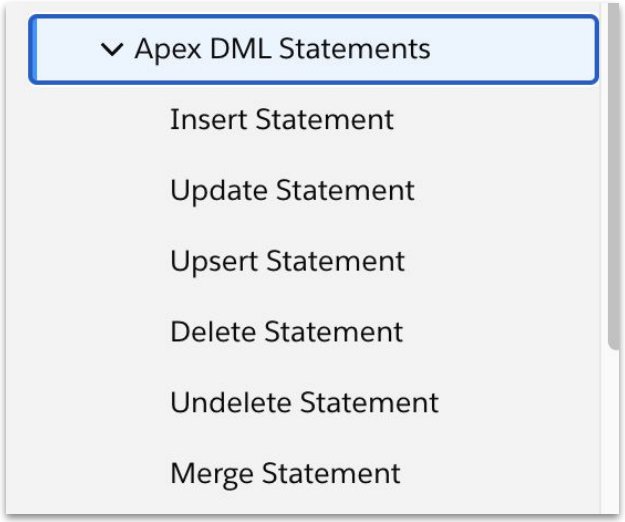
Introduction

Apex is tightly integrated with the Lightning platform persistence layer. Records in the database can be inserted and manipulated through Apex directly using simple statements. The language in Apex that allows you to add and manage records in the database is the Data Manipulation Language (DML).
Before inserting or manipulating records, record data is created in memory as sObjects.

```
Account a = new Account(Name='Account Example');
insert a;

Account a = [SELECT Name,Industry FROM Account WHERE Name='Account Example' LIMIT 1];
// Query existing account.
a.Name = 'Account of the Day';
a.Industry = 'Technology';
// Persist the changes.
update a;

Account a = [SELECT Name,Industry FROM Account WHERE
Name='Account of the Day' LIMIT 1];
// Verify that updated field values were persisted.
System.assertEquals('Account of the Day', a.Name);
System.assertEquals('Technology', a.Industry);
```



Inserting and Updating Records



Using DML, you can insert new records and commit them to the database. Similarly, you can update the field values of existing records.

```
Account[] accts = new List<Account>();
for(Integer i=0;i<3;i++) {
   Account a = new Account(Name='Acme' + i, BillingCity='San Francisco', AccountNumber='0'+i);
   accts.add(a);
}
List<Account> accountToUpdate;
try {
    insert accts;
    accountToUpdate = [SELECT BillingCity FROM Account
                       WHERE Name='Acme2' AND   BillingCity='San Francisco' LIMIT 1];
    accountToUpdate.BillingCity = 'New York';      // Update the billing city in account Acme2.
    update accountToUpdate;          // Make the update call.
} catch(DmlException e) {
   System.debug('An unexpected error has occurred: '  + e.getMessage());
}
// Verify that the billing city was updated to New York.
Account afterUpdate = [SELECT BillingCity FROM Account WHERE Id=:accountToUpdate.Id];
System.assertEquals('New York', afterUpdate.BillingCity);
```

Module 5: Data Manipulation Language

- Insert & Update
- **How DML Works**
  - **Database Class**
  - DML Handling
- Related Records
- Others DMLs
  - Upsert
  - Delete & Undelete



How DML works

You can perform DML operations either on a single sObject, or in bulk on a list of sObjects.

Single vs. Bulk DML Operations

Performing bulk DML operations is the recommended way because it helps avoid hitting governor limits, such as the DML limit of 150 statements per Apex transaction.

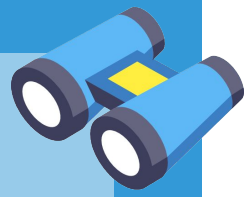
```
for(Contact badCon : conList) {
   if (badCon.Department = 'Finance') {
       badCon.Description__c = 'New
description';
   }
   update badCon;
}
```

Not a good practice since governor limits might be hit

Good practice

```
List<Contact> updatedList = new List<Contact>();
for(Contact con : conList) {
   if (con.Department == 'Finance') {
       con.Description = 'New description';
       // Add updated contact sObject to the list.
       updatedList.add(con);
   }
}
// Call update on the list of contacts.
// This results in one DML call for the entire list.
If(updatedList.size()>0)
   update updatedList;
```

Quick Tour Working with DML Statements



Hands on Lab

1. Insert one record with Apex.
2. Example limit to many DML Statements.
3. Insert more than one record in one transaction.
4. Try insert a record with blank value in required field.
5. Update previous record and try insert a second with blank value in required field.

DML Statements vs. Database Class Methods



Apex offers two ways to perform DML operations: using DML statements or Database class methods.

```
List<Account> acctList = new List<Account>();
acctList.add(new Account(Name='Acme1'));
acctList.add(new Account(Name='Acme2'));
// DML statement
// insert acctList;
// Database Class Methods
Database.SaveResult[] srList = Database.insert(acctList, false);
for (Database.SaveResult sr : srList) {
    if (sr.isSuccess()) {
        System.debug('Successfully inserted. AccountId: ' + sr.getId());
    } else {
        for(Database.Error err : sr.getErrors()) {
            System.debug('The following error has occurred.');
            System.debug(err.getStatusCode() + ': ' + err.getMessage());
            System.debug('Account fields affected: ' + err.getFields());
        }
    }
}
```

DML Statement vs. Database Class Methods II

The ***Database class method***, you can specify whether or not to allow for partial record processing if errors are encountered.

```
Database.SaveResult[] sr = Database.insert(acctList, false);//default true
```



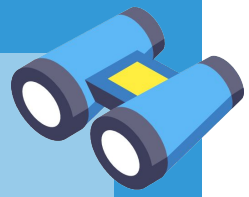
The following helps you decide when you want to use DML statements or Database class methods.

- Use DML statements if you want any error that occurs during bulk DML processing to be thrown as an Apex exception that immediately interrupts control flow (by using try…catch blocks).
- Use Database class methods if you want to allow partial success of a bulk DML operation—if a record fails, the remainder of the DML operation can still succeed. Your application can then inspect the rejected records and possibly retry the operation. When using this form, you can write code that never throws DML exception errors. Instead, your code can use the appropriate results array to judge success or failure.



The convertLead operation is only available as a Database class method, not as a DML statement.
The Database class also provides methods not available as DML statements, such as methods transaction control and rollback, emptying the Recycle Bin, and methods related to SOQL queries.

DML
Working
with class
Database



Hands on Lab

1. Example insert records 20 records with 1 record error with AllorNone = true
2. Example insert records 20 records with 1 record error with AllorNone = false

Module 5: Data Manipulation Language

- Insert & Update
- **How DML Works**
  - Database Class
  - **DML Handling**
- Related Records
- Others DMLs
  - Upsert
  - Delete & Undelete



Exception Handling



While DML statements always return exceptions when an operation fails for one of the records being processed and the operation is rolled back for all records, Database class methods can either do so or allow partial success for record processing. In the latter case of partial processing, Database class methods don't throw exceptions. Instead, they return a list of errors for any errors that occurred on failed records.

```
Account a = new Account(Name='Acme');
try {
    insert a;
} catch(DmlException e) {
  // Process exception here
}
```

Returned Database Errors



Partial inserts can be done with the database.insert class, because methods return a result object that can be used to check which records succeeded, which ones failed, and why.

```
// Create two accounts, one of which is missing a required field
Account[] accts = new List<Account>{
   new Account(Name='Account1'),
   new Account()};
Database.SaveResult[] srList = Database.insert(accts, false);
// Iterate through each returned result
for (Database.SaveResult sr : srList) {
   if (    !sr.isSuccess()) {
   // Operation failed, so get all errors
        for(Database.Error err : sr.getErrors()) {
            System.debug('The following error has occurred.');
            System.debug(err.getStatusCode() + ': ' + err.getMessage());
            System.debug('Fields that affected this error: '  + err.getFields());
        }
   }
}
```

Lab 18: DML Inserting account

Hands on Lab

1. Create an Apex class that inserts a new account named after an input parameter. The account number should be the second input parameter. If the account is successfully inserted, the method should return the account record. If a DML exception occurs, the method should return null.
    - The Apex class must be called 'AccountHandler' and be in the public scope.
    - The Apex class must have a public static method called 'insertNewAccount'.
    - The 'insertNewAccount' method must accept two incoming strings as parameters, name and account number. Insert it into the system and then return the account record.
    - The 'insertNewAccount' method must also accept empty strings, catch the failed DML and return null.

Module 5: Data Manipulation Language

- Insert & Update
- How DML Works
  - Database Class
  - DML Handling
- **Related Records**
- Others DMLs
  - Upsert
  - Delete & Undelete



Inserting Related Records



You can insert records related to existing records if a relationship has already been defined between the two objects, such as a lookup or master-detail relationship.

```
try {
    Account acct = new Account(Name='SFDC Account', AccountNumber='5678');
    insert acct;
    // Once the account is inserted, the sObject will be populated with an
ID.
    // Get this ID.
    ID acctID = acct.ID;
    // Add a contact to this account.
    Contact con = new Contact(
        FirstName = 'Joe',
        LastName = 'Smith',
        Phone = '415.555.1212',
        AccountId = acctID
    );
    insert con;
} catch (DmlException e) {
    System.debug('An unexpected error has occurred: ' + e.getMessage());
}
```

Relating Records by Using an External ID

Add related records by using a custom external ID field on the parent record. Associating records through the external ID field is an alternative to using the record ID. You can add a related record to another record only if a relationship has been defined for the objects involved, such as a master-detail or lookup relationship.

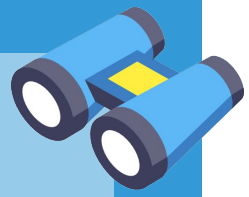
```
Opportunity newOpportunity = new Opportunity(
   Name = 'OpportunityWithAccountInsert',
   StageName = 'Prospecting',
   CloseDate = Date.today().addDays(7)
  );
// Create the parent record reference.
// An account with this external ID value already exists.
// This sObject is used only for foreign key reference and doesn't contain any other fields.
Account accountReference = new Account(MyExtID__c = 'SAP111111');

// Add the nested account sObject to the opportunity using relationshipName.
newOpportunity.Account = accountReference;

Database.insert(newOpportunity);
```

Use of upsert with an external ID can reduce the number of DML statements in your code, and help you to avoid hitting governor limits.

Quick Tour DML Insert related records



Quick Tour

1. Insert Account and related contact using salesforce Id.
2. Query Account and insert a related contact using external Id.

Accessing sObject Fields Through Relationships



sObject records represent relationships to other records with two fields: an ID and an address that points to a representation of the associated sObject.

```
Account a = new Account(Name = 'Acme');
insert a; // Inserting the record automatically assigns a value to its ID field

Contact c = new Contact(LastName = 'Weissman');
c.AccountId = a.Id;
// The new contact now points at the new account
insert c;

// A SOQL query accesses data for the inserted contact,
// including a populated c.account field
c = [SELECT Account.Name FROM Contact WHERE Id = :c.Id];

// Now fields in both records can be changed through the contact
c.Account.Name = 'salesforce.com';
c.LastName = 'Roth';

// To update the database, the two types of records must be updated separately
update c; // This only changes the contact's last name
update c.Account; // This updates the account name
```

Updating Related Records



Fields on related records can't be updated with the same call to the DML operation and require a separate DML call.

```
try {
    // Query for the contact, which has been associated with an account.
    Contact queriedContact = [SELECT Account.Name FROM Contact WHERE FirstName = 'Joe'
                              AND LastName='Smith' LIMIT 1];
    // Update the contact's phone number
    queriedContact.Phone = '415.555.1213';
    // Update the related account industry
    queriedContact.Account.Industry = 'Technology';
    // Make two separate calls
    // 1. This call is to update the contact's phone.
    update queriedContact;
    // 2. This call is to update the related account's Industry field.
    update queriedContact.Account;
} catch(Exception e) {
    System.debug('An unexpected error has occurred: '  + e.getMessage());
}
```

Module 5: Data Manipulation Language

- Insert & Update
- How DML Works
  - Database Class
  - DML Handling
- Related Records
- **Others DMLs**
  - **Upsert**
  - Delete & Undelete



Upserting Records



Using the upsert operation, you can either insert or update an existing record in one call. To determine whether a record already exists, the upsert statement or Database method uses the record's ID as the key to match records, a custom external ID field, or a standard field with the idLookup attribute set to true.

- If the key is not matched, then a new object record is created.
- If the key is matched once, then the existing object record is updated.
- If the key is matched multiple times, then an error is generated and the object record is neither inserted or updated.

```
Account[] acctsList = [SELECT Id, Name, BillingCity FROM Account WHERE BillingCity = 'Bombay'];
for (Account a : acctsList) {
    a.BillingCity = 'Mumbai';
}
Account newAcct = new Account(Name='Acme', BillingCity='San Francisco');
acctsList.add(newAcct);
try {
    upsert acctsList;
} catch (DmlException e) {
    // Process exception here
}
```

Lab 19: DML Upsert records

Hands on Lab

1. Create external Id field in Billing Object
2. In UI create or update a Billing record adding value on the external Id just created.
3. In visual studio, create Apex Class called UpsertBillings.
4. Create a static method without return parameter named upsertBillingList which receives a list of billings with the external id field informed and updates those that already exist and creates the new ones.
5. To test that all is working correctly query created or modified billing in point 2 and change arbitrary value. At the list of billings returned by the query add a new billing with a non existent value on external field.
6. If you query again or see at the list of billing in the UI you must see those billings one modified and other one created.

Module 5: Data Manipulation Language

- Insert & Update
- How DML Works
  - Database Class
  - DML Handling
- Related Records
- **Others DMLs**
  - Upsert
  - **Delete & Undelete**



Deleting & Restoring Records

After you persist records in the database, you can delete those records using the delete operation.

```
Account[] doomedAccts = [
    SELECT Id
    FROM Account
    WHERE Name = 'DotCom'
];
try {
    delete doomedAccts;
} catch (DmlException e) {
    // Process exception here
}
```

Referential Integrity When Deleting and Restoring Records

- The delete operation supports cascading deletions.
- Only for Master-Detail fields

After you have deleted records, the records are placed in the Recycle Bin for 15 days, after which they are permanently deleted.

```
Account a = new Account(Name='Cloud Coachers');
insert(a);
insert(new Contact(LastName='Carter',AccountId=a.Id));
delete a;
Account[] savedAccts = [SELECT Id FROM Account
 WHERE Name = 'Cloud Coachers' AND isDeleted = true
ALL ROWS];
try {
    undelete savedAccts;
} catch (DmlException e) {
    // Process exception here
}
```

Undelete Considerations

- You can undelete records that were deleted as the result of a merge, but the child objects will have been reparented, which cannot be undone.
- Use the **ALL ROWS** parameters with a SOQL query to identify deleted records, including records deleted as a result of a merge.

Lab 20: DML Undelete records

Hands on Lab

1. Query one record of Contact (Only one record).
2. Delete the record in Visual Studio.
3. Go to Recycle bin and check the record was deleted.
4. Recover the record from Recycle bin with apex.

Review Questions

1. A developer in a Salesforce org with 100 accounts executes the following code using the Developer Console.

```
Account myAccount = new Account(Name='MyAccount');
insert myAccount;
for (Integer x = 0; x < 150; x++){
     Account newAccount = new Account(Name='MyAccount' + x);
     try{
          insert newAccount;
     }catch (Exception ex){
          System.Debug(ex);
     }
}
insert new Account(Name='MyAccount');
```

How many accounts are in the org after this code is run?

A. 100
B. 101
C. 102
D. 252



Review Questions

2. Which two statements should a developer avoid using inside procedural loops?

A.
```
List contacts = [select id,firstname,lastname, email
from contact where accountID= :a.Id];
```

B.
```
update contactList;
```

C.
```
if (o.accountid == a.id)
```

D.
```
System.debug('Amount of CPU time (in ms) used so far:
' + Limits.getCpuTime());
```





¡ Gracias !